

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7

дисциплина: Архитектура компьютера —

Студент: Ромеро Гаспар Фернандо Алексей

Группа: НКАбд-02-25

МОСКВА

2025 г.

Оглавление

Цель заботы

Теоретическое введение

2.1 Безусловный переход

2.2 Условные переходы

2.3 Команда сравнения CMP

2.4 Таблица команд условного перехода

Выполнения лабораторной работы

3.2 Программа с условными переходами

3.3 Изучение файла листинга

Выполнение самостоятельной работы

4.1 Нахождение наименьшего из трех чисел

4.2 Вычисление значения функции

Выводы

Список литературы

Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

Теоретическое введение

В архитектуре компьютеров центральный процессор (ЦП) следует счётчику команд (РС), который обычно увеличивается последовательно, указывая на следующую инструкцию в памяти. Это линейное выполнение прерывается инструкциями перехода (jumps или branches), которые являются основополагающими для реализации высокоуровневых управляющих структур, таких как условные конструкции и циклы. Эти инструкции изменяют значение РС, позволяя процессору передавать управление на другой адрес в памяти — либо безусловно, либо условно, на основе состояния флагов процессора.

Передачи управления позволяют отклоняться от последовательного потока выполнения и необходимы для принятия решений и повторения кода. В языке ассемблера это достигается в основном с помощью двух типов инструкций: 1) Условные переходы (например, JE, JNE, JG в x86), которые оценивают условия и передают управление, если выполняется определённый критерий, и 2) Безусловные переходы (например, JMP), которые всегда перенаправляют выполнение на целевую адрес. Кроме того, вызовы подпрограмм (CALL) и инструкции возврата (RET) также представляют собой механизм передачи управления для модульности программы.

2.1. Безусловный переход

Безусловный переход — это машинная инструкция, которая принудительно заставляет процессор продолжить выполнение с другого адреса памяти, не завися от флагов или условий. В ассемблере x86/x64 инструкция JMP место_назначения напрямую загружает значение назначения в регистр RIP/EIP (указатель инструкций), прерывая последовательный поток выполнения. Место назначения может быть меткой внутри того же сегмента кода (относительный переход, кодируемый как смещение на 8, 16 или 32 бита) или абсолютным значением, хранящимся в регистре или памяти (например, JMP EAX). Это действие является немедленным и детерминированным

Его основное применение — реализация циклов и пропуск кода. Например, в цикле `while(1)` компилятор генерирует:

```
метка_начала:  
    ; Тело цикла  
    JMP метка_начала ; Безусловный переход к началу
```

Он также используется для выхода из условных структур: после выполнения блока `if`, инструкция `JMP` переходит к концу блока `else`, чтобы избежать его выполнения. Это фундаментальный механизм, позволяющий компиляторам переводить высокоуровневые структуры (C, Java) в машинный код.

2.2. Условные переходы

Условные переходы — это инструкции передачи управления, которые выполняют отклонение только при выполнении определённого логического условия, оцениваемого с помощью флагов регистра состояния (EFLAGS/RFLAGS). В x86/x64 инструкции, такие как `'JE'` (Jump if Equal — переход, если равно), `'JNE'` (Jump if Not Equal — переход, если не равно), `'JG'` (Jump if Greater — переход, если больше) или `'JC'` (Jump if Carry — переход, если есть перенос), проверяют флаги, такие как `'ZF'` (Zero Flag — флаг нуля), `'CF'` (Carry Flag — флаг переноса) или `'SF'` (Sign Flag — флаг знака). Процессор принимает решение во время выполнения: если условие истинно, он загружает целевой адрес в RIP/EIP; если ложно, продолжает выполнение следующей последовательной инструкции.

Эти инструкции напрямую переводят структуры `if-else`, `while` и `for`. Например, `CMP EAX, 5`, за которым следует `JGE` метка, реализует `if (x >= 5)`. Их критическая производительность зависит от **предсказателя переходов** процессора. Поскольку условие разрешается поздно в конвейере, процессор предсказывает, будет ли переход выполнен. Неверное предсказание заставляет процессор отбрасывать уже обработанные инструкции (сброс конвейера), вызывая штраф в 10-20 тактов. Именно поэтому компиляторы оптимизируют порядок ветвлений (размещая наиболее вероятный путь без перехода).

2.3. Команда сравнения CMP

Инструкция `'CMP'` выполняет вычитание в арифметико-логическом устройстве (АЛУ), где `'CMP A, B'` вычисляет `A - B`. Результат не записывается в какой-либо регистр, но немедленно обновляет шесть флагов в EFLAGS/RFLAGS: `'CF'` (заём/borrow при вычитании), `'PF'` (чётность), `'AF'` (коррекция), `'ZF'` (ноль), `'SF'` (знак результата) и `'OF'` (переполнение со знаком). Например, `'CMP EAX, 0x80'` устанавливает `'ZF=1'` только в том случае, если `'EAX'` содержит ровно 128 в десятичной системе. Эта операция занимает 1 такт и позволяет

выполнять сравнения для 8, 16, 32 или 64 бит.

Каждая инструкция условного перехода проверяет определенную комбинацию флагов, установленных операцией `CMP`. После `CMP A, B`:

- * `JE` переходит, если `ZF=1` ($A = B$)
- * `JA` переходит, если `CF=0` и `ZF=0` (без знака: $A > B$)
- * `JG` переходит, если `SF=OF` и `ZF=0` (со знаком: $A > B$)

Компиляторы оптимизируют распространённые последовательности; `CMP REG, 0`, за которым следует `JZ`, эквивалентно `TEST REG, REG`, за которым следует `JZ`, но `CMP` необходим для сравнений с ненулевыми константами.

2.4. Таблица команд условного перехода

ИНСТРУКЦИЯ	УСЛОВИЕ	ПРОВЕРЯЕМЫЕ ФЛАГИ	ОПИСАНИЕ	ЭКВИВАЛЕНТНОЕ УСЛОВИЕ (ПОСЛЕ CMP A,B)
JE / JZ	Равно / Ноль	ZF = 1	Переход, если равно/ноль	$A == B$
JNE / JNZ	Не равно / Не ноль	ZF = 0	Переход, если не равно/не ноль	$A != B$
JC / JB / JNAE	Перенос / Ниже / Не выше или равно	CF = 1	Переход, если перенос/ниже (без знака)	$A < B$ (без знака)
JNC / JAE / JNB	Нет переноса / Выше или равно / Не ниже	CF = 0	Переход, если нет переноса/выше или равно (без знака)	$A >= B$ (без знака)
JA / JNBE	Выше / Не ниже или равно	CF = 0 И ZF = 0	Переход, если выше (без знака)	$A > B$ (без знака)
JBE / JNA	Ниже или равно / Не выше	CF = 1 ИЛИ ZF = 1	Переход, если ниже или равно (без знака)	$A <= B$ (без знака)
JG / JNLE	Больше / Не меньше или равно	ZF = 0 И SF = OF	Переход, если больше (со знаком)	$A > B$ (со знаком)

JGE / JNL	Больше или равно / Не меньше	SF = OF	Переход, если больше или равно (со знаком)	A >= B (со знаком)
JL / JNGE	Меньше больше / Не равно	SF != OF	Переход, если меньше (со знаком)	A < B (со знаком)
JLE / JNG	Меньше равно / Не больше	ZF = 1 ИЛИ SF != OF	Переход, если меньше или равно (со знаком)	A <= B (со знаком)
JS	Знак (отрицательный)	SF = 1	Переход, если отрицательный	Результат отрицательный
JNS	Нет знака (не отрицательный)	SF = 0	Переход, если не отрицательный	Результат отрицательный
JO	Переполнение	OF = 1	Переход, если переполнение	Арифметическое переполнение
JP / JPE	Чётность / Чётная чётность	PF = 1	Переход, если чётное число единичных битов	Результат с чётным числом битов 1
JNP / JPO	Нечётность / Нечётная чётность	PF = 0	Переход, если нечётное число единичных битов	Результат с нечётным числом битов 1
LOOP	Цикл	ECX != 0	Уменьшает ECX и переход, если не ноль	Для циклов со счётчиком
JCXZ / JECXZ / JRCXZ	Переход, если CX/ECX/RCX ноль	CX/ECX/RCX = 0	Переход, если регистр-счётчик равен нулю	Проверка счётчика ноль
JNO	Нет переполнения	OF = 0	Переход, если нет переполнения	Без переполнения

Выполнение лабораторной работы

Для начала создайте в рабочей директории каталог с именем lab07 и перейдите в него. Затем создайте файл lab7-1.asm с помощью текстового редактора.

Задача состоит в написании программы, которая использует инструкцию jmp для изменения порядка сообщений, отображаемых на экране.


```
fernando@ubuntu25-10:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
fernando@ubuntu25-10:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1
```

3.2 Программа с условными переходами

Вторая задача требует написания программы, которая определяет наибольшее из трех чисел: переменные А и С определены в коде, а переменная В вводится с клавиатуры.

Создайте файл lab7-2.asm. Для реализации ввода и сравнения чисел я использовал функции из включенного файла in_out.asm. Обратите внимание, что ввод с клавиатуры приводит к получению строки, которую необходимо преобразовать в целое число с помощью функции atoi перед выполнением арифметического сравнения.

Алгоритм программы:

1. Программа выводит сообщение "Введите В".
2. Она считывает строку с консоли.
3. Она преобразует введенную строку в число (используя функцию `atoi`).
4. Она сравнивает число А (по умолчанию) с числом С. Если $A > C$, программа записывает А как текущее максимальное значение; в противном случае она записывает С.
5. Текущее максимальное значение сравнивается с введенным числом В. Если В больше, оно становится новым максимальным значением.
6. Результат отображается на экране.

```
fernando@ubuntu25-10:~/work/arch-pc/lab07$ touch lab7-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab07$ nano lab7-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
fernando@ubuntu25-10:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 7
Наибольшее число: 50
```

3.3 Изучение файла листинга

Листинг-файл — это текстовый файл, созданный ассемблером, который показывает взаимосвязь между исходным кодом и машинными инструкциями. Создать листинг-файл можно с помощью команды:

```
fernando@ubuntu25-10:~/work/arch-pc/lab07$ nasm -f elf -l lab7-2.lst lab7-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab07$
```

Затем откройте полученный файл lab7-2.lst в текстовом редакторе. Вот фрагмент содержимого файла:

```
31 0000011C 3B0D[3A000000]    cmp ecx,[C]      ; Сравниваем 'A' и 'C'
32 00000122 7F0C                jg check_B      ; если 'A>C', то переход на метку 'check_B',
```

Разберем одну из строк листинга подробно:

- 31, 32 — это номера строк в исходном коде .asm.
- 0000011C, 00000122 — это смещения (адреса) команд от начала сегмента кода в шестнадцатеричном формате.
- 3B0D[3A000000] — машинный код команды `cmp ecx, [C]`.
- 7F0C — машинный код команды `jg check_B`. Здесь 7F — это код операции (опкод) перехода, а 0C — это операнд, указывающий, на сколько байт нужно прыгнуть вперед.

Эксперимент с ошибкой: При попытке удалить один из операндов из инструкции `cmp` и повторной компиляции с опцией `-l` ассемблер вернул в терминале ошибку: «Недопустимая комбинация кода операции и операндов». Однако файл листинга не был создан. Это демонстрирует, что листинг генерируется только при успешной компиляции кода.

Задания для самостоятельной работы

Задание 4.1. Нахождение наименьшего из трех чисел

Вариант задания: 11

1. Напишите программу нахождения наименьшей из 3 целочисленных переменных a, b и c . Значения переменных выбрать из табл. 7.5 в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу.

Номер варианта	Значения a, b, c
11	21,28,34

```
fernando@ubuntu25-10:~/work/arch-pc/lab07$ touch lab7-3.asm
fernando@ubuntu25-10:~/work/arch-pc/lab07$ nano lab7-3.asm
fernando@ubuntu25-10:~/work/arch-pc/lab07$ nasm -f elf lab7-3.asm
fernando@ubuntu25-10:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-3 lab7-3.o
fernando@ubuntu25-10:~/work/arch-pc/lab07$ ./lab7-3
El menor número es: 21
```

Задание 4.2. Вычисление значения функции

2. Напишите программу, которая для введенных с клавиатуры значений x и a вычисляет значение заданной функции $f(x)$ и выводит результат вычислений. Вид функции $f(x)$ выбрать из таблицы 7.6 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу для значений x и a из 7.6

Номер варианта	Выражение для $f(x)$	$(x1, a1)$	$(x2, a2)$
11	$f(x) = \begin{cases} 4a, & x = 0 \\ 4a + x, & x \neq 0 \end{cases}$	(0;3)	(1;2)

Вам необходимо загрузить файл in_out.asm, скопировать его в свою рабочую директорию, создать копию файла lab5-1.asm под названием lab5-2.asm и изменить её, чтобы она использовала процедуры из файла in_out.asm.

```
fernando@ubuntu25-10:~/work/arch-pc/lab07$ touch lab7-4.asm
fernando@ubuntu25-10:~/work/arch-pc/lab07$ nano lab7-4.asm
fernando@ubuntu25-10:~/work/arch-pc/lab07$ nasm -f elf lab7-4.asm
fernando@ubuntu25-10:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-4 lab7-4.o
fernando@ubuntu25-10:~/work/arch-pc/lab07$ ./lab7-4
Введите x: 1
Введите a: 4
Результат f(x) = 17
fernando@ubuntu25-10:~/work/arch-pc/lab07$ ./lab7-4
Введите x: 1
Введите a: 2
Результат f(x) = 9
```

Выводы

В ходе лабораторной работы №7 изучались механизмы управления потоком выполнения программ в архитектуре i386 с использованием ассемблера NASM.

Были выполнены следующие задачи:

1. Изучены теоретические основы ветвления: функция регистра флагов (EFLAGS) и работа инструкций перехода.
2. Освоена инструкция безусловного перехода `jmp`. Установлено, что её можно использовать для произвольного изменения порядка выполнения инструкций, создавая «спагетти-код» или, наоборот, структурированные циклы.
3. Реализованы программы с условными переходами. Студенты научились комбинировать инструкцию сравнения `cmp` с инструкциями `jl`, `jg` и `jge` для реализации логики `if-else`. Это позволило им решать такие задачи, как нахождение максимума и минимума чисел, а также вычисление кусочно-заданной функции.
4. Проанализирован листинг-файл (`.lst`). Было обнаружено, что листинг является мощным инструментом отладки, позволяющим просматривать машинный код и проверять правильность перевода меток и адресов ветвлений. Эксперименты подтвердили, что листинг генерируется только в том случае, если в коде нет синтаксических ошибок.

Таким образом, цель данной работы достигнута: приобрести навыки программирования ветвлений на языке ассемблера, что является основой для реализации любого сложного алгоритма на низком уровне.

Список литературы

1. Intel Corporation. (2023). *Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2: Instruction Set Reference*.
<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
2. Patterson, D. A., & Hennessy, J. L. (2017). *Computer Organization and Design: The Hardware/Software Interface* (5th ed.). Morgan Kaufmann.